TRƯỜNG ĐẠI HỌC CNTT VÀ TRUYỀN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO

HỌC PHẦN TRIỂN KHAI & QUẢN TRỊ HỆ THỐNG PHẦN MỀM

Đề tài:

Website Quảng bá Sản phẩm (WordPress)

Sinh viên thực hiện: **PHÙNG DUY HOÀNG**

Mã sinh viên: **DTC245230006**

Lớp: **HTTT K232A**

Giảng viên hướng dẫn: **Hoàng Văn Tâm**

THÁI NGUYÊN, NĂM 2026

MỤC LỤC

LỜI NÓI ĐẦU 4
PHẦN 1: CƠ SỞ LÝ THUYẾT, KIẾN TRÚC VÀ PHÂN TÍCH YÊU CẦU 6
1.1. Bối cảnh, mục tiêu và yêu cầu của đề tài 6
1.2. Kiến trúc hệ thống và môi trường triển khai 7
1.3. Phân tích nghiệp vụ và mô hình dữ liệu WordPress 8
PHẦN 2: XÂY DỰNG, ĐÓNG GÓI VÀ TRIỂN KHAI ỨNG DỤNG 10
2.1. Cấu trúc mã nguồn và các mốc phát hành 10
2.2. Đóng gói ứng dụng bằng Docker Compose 11
2.3. Cơ sở dữ liệu MySQL, phpMyAdmin và cài đặt website 12
2.3.1. Cấu hình MySQL 12
2.3.2. Nguyên tắc phân quyền tối thiểu 13
2.3.3. phpMyAdmin 13
2.3.4. Cài đặt WordPress không cần trình duyệt 13
2.4. Kiểm thử tự động và pipeline CI/CD 14
2.5. Triển khai, kiểm thử và xử lý sự cố thực tế 15
PHẦN 3: TRIỂN KHAI NGINX, HTTPS, BẢO MẬT VÀ SAO LƯU 17
3.1. Nginx reverse proxy và HTTPS 17
3.2. Tăng cường bảo mật máy chủ và ứng dụng 19
3.3. Sao lưu và phục hồi 20
PHẦN 4: GIÁM SÁT VÀ THU THẬP LOG TẬP TRUNG 22
4.1. Prometheus và các chỉ số thu thập 22
4.2. Grafana và dashboard 23
4.3. Thu thập log tập trung với Loki và Promtail 24
4.4. Đánh giá hệ thống giám sát và kết quả nghiệm thu 26

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 28
TÀI LIỆU THAM KHẢO 30

LỜI NÓI ĐẦU

Trong bối cảnh thương mại điện tử phát triển mạnh, website không còn chỉ là trang thông tin giới thiệu mà trở thành kênh bán hàng và xây dựng thương hiệu chính của doanh nghiệp. Một website quảng bá sản phẩm tốt cần giới thiệu được danh mục, hình ảnh, thông số kỹ thuật và câu chuyện sản phẩm; đồng thời phải tải nhanh, hoạt động ổn định, có thể cập nhật nội dung thường xuyên mà không cần lập trình viên. Với các đơn vị vừa và nhỏ, WordPress là lựa chọn phổ biến vì mã nguồn mở, hệ sinh thái giao diện và plugin phong phú, cộng đồng hỗ trợ lớn.

Tuy nhiên, đưa WordPress lên môi trường thật đòi hỏi nhiều hơn việc cài đặt vài cú nhấp chuột: cần tách biệt tầng ứng dụng và tầng dữ liệu, chấm dứt TLS tại một reverse proxy, đóng gói để tái lập được, tự động kiểm tra chất lượng mã nguồn, sao lưu định kỳ và đặc biệt là quan sát được hệ thống đang chạy ra sao. Trong học phần Triển khai và Quản trị Hệ thống Phần mềm, Đề 3 yêu cầu thực hiện đầy đủ các nội dung đó trên một máy ảo Linux thật.

Báo cáo trình bày kết quả triển khai hệ thống Website Quảng bá Sản phẩm bằng WordPress trên máy ảo Ubuntu 18.04.6 LTS tại địa chỉ 192.168.159.128, sử dụng Docker Compose để điều phối mười container: Nginx, WordPress, MySQL, phpMyAdmin, Prometheus, Grafana, Loki, Promtail, node-exporter và mysqld-exporter. Mã nguồn được quản lý bằng Git và đóng gói bằng GitHub Actions; toàn bộ cấu hình bảo mật, sao lưu và giám sát đều ở dạng tệp cấu hình có thể kiểm tra, tái sử dụng. Các số liệu trong báo cáo được thu thập trực tiếp trên máy ảo tại thời điểm nghiệm thu ngày 25/10/2026.

Nội dung báo cáo gồm bốn phần. Phần 1 trình bày cơ sở lý thuyết, kiến trúc và phân tích yêu cầu. Phần 2 trình bày đóng gói, quản trị cơ sở dữ liệu, cài đặt website, kiểm thử tự động và triển khai. Phần 3 trình bày Nginx, HTTPS, tăng cường bảo mật và sao lưu phục hồi. Phần 4 trình bày giám sát với Prometheus – Grafana và thu thập log tập trung với Loki – Promtail, kèm đánh giá kết quả đo được. Em xin trân trọng cảm ơn thầy giáo hướng dẫn và các thầy cô Khoa Công nghệ thông tin đã truyền đạt kiến

thức và góp ý trong suốt học phần; báo cáo khó tránh khỏi thiếu sót, em mong nhận được nhận xét để tiếp tục hoàn thiện.

PHẦN 1: CƠ SỞ LÝ THUYẾT, KIẾN TRÚC VÀ PHÂN TÍCH YÊU CẦU

1.1. Bối cảnh, mục tiêu và yêu cầu của đề tài

Các doanh nghiệp sản xuất và phân phối hàng tiêu dùng thường gặp ba khó khăn khi xây dựng kênh bán hàng trực tuyến: chi phí thuê ngoài dịch vụ thiết kế website cao và khó thay đổi; nội dung sản phẩm cập nhật chậm vì phụ thuộc đơn vị kỹ thuật; và việc vận hành thiếu công cụ theo dõi khiến sự cố (hết dung lượng, quá tải cơ sở dữ liệu, chứng chỉ hết hạn) chỉ được phát hiện khi khách đã phản ánh. Đề tài hướng tới một hệ thống tự vận hành được, chi phí thấp, nhân viên nội dung có thể tự đăng sản phẩm và bộ phận kỹ thuật có dữ liệu để chủ động xử lý.

Đề 3 đặt ra các yêu cầu kỹ thuật bắt buộc sau:

- **Website giới thiệu sản phẩm** xây dựng trên WordPress (hoặc tương đương), có giao diện công khai và trang quản trị nội dung.
- **Cơ sở dữ liệu MySQL** kèm **phpMyAdmin** để quản trị trực quan.
- **Quản lý mã nguồn trên GitHub**, có quy trình tích hợp liên tục kiểm tra cấu hình trước khi triển khai.
- **Nginx làm reverse proxy**, chấm dứt HTTPS và định tuyến tới các dịch vụ nội bộ.
- **Prometheus và Grafana** thu thập, trực quan hoá chỉ số hệ thống và ứng dụng.
- **Loki và Promtail** thu thập log tập trung, truy vấn bằng LogQL.
- **Hardening**: giảm bề mặt tấn công, phân quyền tối thiểu, không lộ cổng dữ liệu ra ngoài, bảo vệ bí mật cấu hình.

Trên cơ sở đó, đề tài đặt mục tiêu cụ thể: (i) website hoạt động qua HTTPS với đầy đủ chức năng xem và quản trị nội dung; (ii) toàn bộ hệ thống tái lập được bằng một lệnh `docker compose up -d` từ kho mã nguồn; (iii) có dashboard giám sát hiển thị CPU, RAM, dung lượng đĩa, kết nối MySQL và log Nginx theo thời gian thực; (iv) có quy trình sao lưu, phục hồi và kiểm tra sức khoẻ hệ thống tự động.

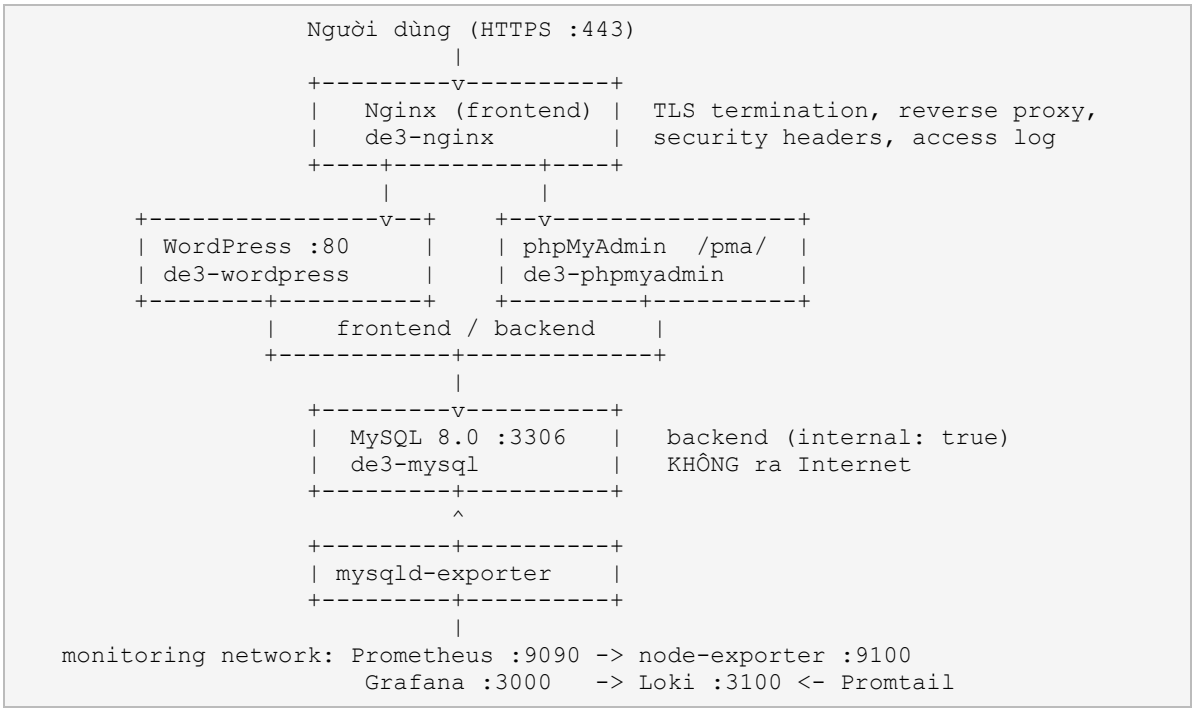
1.2. Kiến trúc hệ thống và môi trường triển khai

Hệ thống được triển khai trên một máy ảo Linux đóng vai trò máy chủ đơn (single host). Toàn bộ thành phần chạy dưới dạng container, điều phối bởi Docker Compose. Thông tin môi trường được tổng hợp trong Bảng 1.1.

Bảng 1.1. Môi trường triển khai

Thành phần	Thông số
Hệ điều hành	Ubuntu 18.04.6 LTS, kernel 4.15.0-213-generic
Địa chỉ máy ảo	192.168.159.128 (SSH cổng 22, xác thực bằng khoá ED25519)
Tài nguyên	2 vCPU, 4,1 GB RAM, ổ đĩa 20 GB (đã dùng 63% tại thời điểm nghiệm thu)
Docker Engine	20.10.21, build 20.10.21-0ubuntu1~18.04.3
Docker Compose	v2.39.2 (plugin, dùng lệnh `docker compose`)
Thư mục triển khai	`/home/thinh/de3-wordpress-quangba`
Tên miền nội bộ	`shop.local` (phân giải qua `--resolve` khi kiểm thử)

Kiến trúc logic chia làm ba vùng mạng độc lập, thể hiện nguyên tắc tách biệt tầng và giảm bề mặt tấn công:



Bảng 1.2. Mười dịch vụ trong hệ thống

#	Container	Image	RAM	Vai trò và mạng tham gia
1	de3-nginx	de3-nginx:1.0 (build từ nginx:alpine)	128 MB	Reverse proxy, TLS, cổng 80/443 ra ngoài — frontend

2	de3-wordpress	wordpress:php8.2-apache	512 MB	Ứng dụng website — frontend, backend
3	de3-mysql	mysql:8.0	768 MB	Cơ sở dữ liệu, không publish cổng — backend
4	de3-phpmyadmin	phpmyadmin:latest	256 MB	Quản trị CSDL qua web — frontend, backend
5	de3-prometheus	prom/prometheus:latest (3.14.0)	512 MB	Thu thập metrics, bind 127.0.0.1:9090 — monitoring
6	de3-grafana	grafana/grafana:latest (13.2.2)	384 MB	Dashboard, phục vụ tại /grafana/ — frontend, monitoring
7	de3-loki	grafana/loki:latest (3.7.8)	384 MB	Lưu trữ log tập trung — monitoring
8	de3-promtail	grafana/promtail:latest	128 MB	Đọc log container, đẩy về Loki — monitoring
9	de3-node-exporter	prom/node-exporter:latest	64 MB	Metrics CPU/RAM/đĩa mạng của host — monitoring
10	de3-mysqld-exporter	prom/mysqld-exporter:latest	64 MB	Metrics MySQL — backend, monitoring

Bảng 1.3. Ba mạng Docker và mục đích cách ly

Mạng	Loại	Thành viên và lý do
frontend	bridge	nginx, wordpress, phpmyadmin, grafana. Chỉ các dịch vụ cần nhận request từ Nginx mới nằm ở đây.
backend	bridge, **internal: true**	mysql, wordpress, phpmyadmin, mysqld-exporter. Không có route ra Internet, nên dù bị chiếm quyền, kẻ tấn công không thể đưa dữ liệu ra ngoài qua tầng này.
monitoring	bridge	prometheus, grafana, loki, promtail, node-exporter, mysqld-exporter. Tách lưu lượng giám sát khỏi lưu lượng người dùng.

Sáu volume có tên được khai báo để dữ liệu tồn tại độc lập với vòng đời container: `mysql_data`, `wordpress_data`, `prometheus_data`, `grafana_data`, `loki_data` và `nginx_logs`. Nhờ đó có thể nâng cấp image, tạo lại container mà không mất cơ sở dữ liệu, nội dung đã đăng hay lịch sử chỉ số.

1.3. Phân tích nghiệp vụ và mô hình dữ liệu WordPress

Về nghiệp vụ, hệ thống phục vụ hai nhóm người dùng. Nhóm công khai gồm khách truy cập: xem trang chủ, danh mục sản phẩm, trang chi tiết sản phẩm, tin tức và gửi liên hệ. Nhóm quản trị gồm biên tập viên và quản trị viên: đăng/sửa sản phẩm, tải ảnh, quản lý danh mục, bình luận và cấu hình website. WordPress đáp ứng sẵn các nghiệp vụ này thông qua mô hình nội dung gồm bài viết, trang và danh mục.

Cơ sở dữ liệu do WordPress tự sinh trong lần cài đặt đầu tiên, gồm các bảng cốt lõi được mô tả trong Bảng 1.4. Việc hiểu cấu trúc này cần thiết cho công tác sao lưu, tối ưu truy vấn và tạo tài khoản chỉ đọc phục vụ báo cáo.

Bảng 1.4. Các bảng cốt lõi của WordPress

Bảng	Nội dung
wp_posts	Bài viết, trang, sản phẩm, bản nháp, menu, media. Trường `post_type` phân biệt loại nội dung, `post_status` phân biệt công khai/bản nháp.
wp_postmeta	Siêu dữ liệu gắn với bài viết: giá, mã sản phẩm, ảnh phụ, thông số kỹ thuật (khoá – giá trị).
wp_terms / wp_term_taxonomy	Danh mục, thẻ và các taxonomy; phân cấp danh mục sản phẩm.
wp_term_relationships	Liên kết nhiều–nhiều giữa nội dung và danh mục.
wp_users / wp_usermeta	Tài khoản quản trị, biên tập viên; mật khẩu lưu dưới dạng băm (bcrypt/phpass), không lưu văn bản thuần.
wp_options	Cấu hình website: tên miền, đường dẫn, giao diện đang dùng, plugin kích hoạt.
wp_comments	Bình luận của khách, cần kiểm duyệt để tránh spam.

Vì mục tiêu của đề tài là *quảng bá* sản phẩm chứ không phải bán hàng có giỏ hàng và thanh toán, hệ thống sử dụng mô hình nội dung mặc định của WordPress: mỗi sản phẩm là một bài viết/trang có ảnh đại diện, mô tả và thông số trong `wp_postmeta`. Cách tiếp cận này giữ hệ thống nhẹ, phù hợp giới hạn 4,1 GB RAM của máy ảo. Nếu cần giỏ hàng thật, có thể bổ sung WooCommerce mà không phải thay đổi kiến trúc hạ tầng; phần này được nêu trong hướng phát triển ở cuối báo cáo.

Về yêu cầu phi chức năng, hệ thống đặt ra các chỉ tiêu: thời gian phản hồi trang dưới 1 giây với nội dung tĩnh; độ sẵn sàng mục tiêu 99% trong giờ học; sao lưu hằng ngày và giữ 7 bản gần nhất; log được lưu tối thiểu 7 ngày để phục vụ điều tra sự cố.

PHẦN 2: XÂY DỰNG, ĐÓNG GÓI VÀ TRIỂN KHAI ỨNG DỤNG

2.1. Cấu trúc mã nguồn và các mốc phát hành

Mã nguồn được tổ chức theo nguyên tắc "hạ tầng là mã" (Infrastructure as Code): mọi cấu hình đều là tệp văn bản nằm trong kho Git, không có bước thiết lập thủ công nào trên máy chủ. Cấu trúc thư mục dự án như sau:

```
de3-wordpress-quangba/
├── docker-compose.yml          # Điều phối 10 dịch vụ, 3 mạng, 6 volume
├── .env.example                # Mẫu biến môi trường (không chứa giá trị thật)
├── .gitignore                  # Loại trừ .env, khoá riêng tư, bản sao lưu
├── .gitattributes              # Ép LF cho script Linux, chặn CRLF
├── README.md                   # Kiến trúc, hướng dẫn triển khai, PromQL/LogQL
mẫu
├── nginx/
│   ├── Dockerfile              # nginx:alpine + cấu hình riêng + healthcheck
│   ├── nginx.conf              # server_tokens off, log format, gzip
│   ├── templates/default.conf.template   # Sinh bằng envsubst khi khởi động
│   └── certs/                  # server.crt, server.key (bị .gitignore)
├── mysql/
│   ├── conf.d/my.cnf           # utf8mb4, slow query log, buffer pool
│   └── init/01-create-users.sh # Tạo user chỉ đọc và user cho exporter
├── prometheus/
│   ├── prometheus.yml          # 4 scrape job
│   └── rules/alerts.yml        # 3 nhóm, 8 cảnh báo
├── loki/loki-config.yml        # Schema v13/tsdb, retention 168h
├── promtail/promtail-config.yml# docker_sd_configs + relabel
├── grafana/provisioning/
│   ├── datasources/datasources.yml      # UID cố định prometheus-main, loki-
main
│   └── dashboards/                      # Provider + de3-overview.json
├── mysqld-exporter/.my.cnf     # Chứa mật khẩu (bị .gitignore, chmod 600)
├── scripts/
│   ├── init-env.sh             # Sinh .env và .my.cnf với mật khẩu ngẫu nhiên
│   ├── generate-certs.sh       # Tự ký chứng chỉ TLS kèm SAN
│   ├── install-wordpress.sh    # Cài WordPress qua HTTP, không cần trình duyệt
│   ├── deploy.sh               # Triển khai một lệnh
│   ├── healthcheck.sh          # Kiểm tra 6 endpoint + Prometheus targets
│   ├── verify-observability.sh # Minh chứng Prometheus/Grafana/Loki
│   ├── backup.sh / restore.sh  # Sao lưu và phục hồi
│   └── diagnose.sh             # Thu thập thông tin khi có sự cố
└── .github/workflows/ci.yml    # Pipeline kiểm tra tự động
```

Kho mã nguồn ghi nhận ba mốc phát hành chính, thể hiện quá trình hoàn thiện dần của hệ thống:

Bảng 2.1. Các mốc phát hành trong Git

Commit	Nội dung
db7734b	Khởi tạo dự án: docker-compose, cấu hình Nginx, MySQL, Prometheus, Grafana, Loki và bộ script cơ bản.
c365c37	Thêm `.gitattributes` buộc xuống dòng LF cho script Linux — sửa lỗi script không chạy trên máy ảo do bị chuyển sang CRLF khi soạn trên Windows.

75906cb	Sửa ba lỗi triển khai thực tế (mysqld-exporter, Nginx giữ DNS cũ, Grafana provisioning) và bổ sung script cài đặt, kiểm tra, chẩn đoán.

2.2. Đóng gói ứng dụng bằng Docker Compose

Tệp `docker-compose.yml` khai báo mười dịch vụ. Để tránh lặp lại cấu hình, phần chung được gom vào một YAML anchor `x-common` rồi hợp nhất vào từng dịch vụ bằng `<<: *common`:

```
x-common: &common
  restart: unless-stopped
  security_opt:
    - no-new-privileges:true
  logging:
    driver: json-file
    options:
      max-size: "10m"
      max-file: "3"
```

Ba thiết lập trong khối chung đều có mục đích vận hành rõ ràng. `restart: unless-stopped` giúp container tự khởi động lại sau khi máy ảo reboot hoặc khi tiến trình chính chết, nhưng vẫn tôn trọng quyết định dừng thủ công của người quản trị. `no-new-privileges` chặn việc leo thang đặc quyền qua setuid/setgid bên trong container. Giới hạn log `max-size 10m` và `max-file 3` ngăn một container lỗi liên tục ghi log làm đầy ổ đĩa — đây là sự cố đã gặp trên máy ảo 20 GB trước khi áp dụng.

Mỗi dịch vụ có `mem_limit` riêng, tổng khoảng 3,2 GB trên 4,1 GB RAM, phần còn lại dành cho hệ điều hành và bộ đệm đĩa. Việc đặt giới hạn giúp một thành phần bị rò rỉ bộ nhớ không kéo sập toàn hệ thống; kernel sẽ dừng riêng container đó và cơ chế restart đưa nó trở lại.

Thứ tự khởi động được kiểm soát bằng `depends_on` kèm điều kiện dựa trên healthcheck thay vì chỉ dựa vào thứ tự tạo container. MySQL được coi là sẵn sàng khi `mysqladmin ping` trả lời; WordPress chỉ khởi động sau đó; Nginx chỉ khởi động khi WordPress đã qua healthcheck:

```
depends_on:
  wordpress:
    condition: service_healthy
  phpmyadmin:
    condition: service_started
  grafana:
    condition: service_started
```

Healthcheck của WordPress dùng PHP có sẵn trong image để mở socket tới cổng 80, tránh phải cài thêm công cụ: `php -r "exit(@fsockopen('127.0.0.1',80)?0:1);"`. Với Nginx, image alpine có sẵn `wget` nên kiểm tra trực tiếp endpoint `/healthz` qua HTTPS:

```
healthcheck:
  test: ["CMD", "wget", "-q", "--no-check-certificate", "-O", "/dev/null",
"https://localhost/healthz"]
  interval: 30s
  timeout: 5s
  retries: 3
  start_period: 15s
```

Ở lần triển khai đầu, WordPress cần khoảng 60–90 giây để giải nén mã nguồn và tạo tệp cấu hình, nên `start_period` được đặt 90 giây. Nếu không có tham số này, healthcheck sẽ báo lỗi giả trong lúc ứng dụng còn đang khởi tạo và Nginx không được phép khởi động.

2.3. Cơ sở dữ liệu MySQL, phpMyAdmin và cài đặt website

2.3.1. Cấu hình MySQL

MySQL 8.0 chạy với các tham số được đặt qua `command` để bảo đảm tiếng Việt hiển thị đúng và giới hạn tài nguyên hợp lý:

```
command:
  - --default-authentication-plugin=caching_sha2_password
  - --character-set-server=utf8mb4
  - --collation-server=utf8mb4_unicode_ci
  - --max-connections=100
```

`utf8mb4` cần thiết vì nội dung sản phẩm tiếng Việt và emoji cần 4 byte cho một số ký tự; dùng `utf8` cũ sẽ làm hỏng dữ liệu. `max-connections=100` đủ cho lưu lượng học tập và tránh cạn bộ nhớ do quá nhiều kết nối.

Tệp `mysql/conf.d/my.cnf` bổ sung cấu hình phía máy chủ: bật `slow_query_log` với ngưỡng 2 giây để phát hiện truy vấn chậm, đặt `innodb_buffer_pool_size = 256M` phù hợp với 768 MB cấp cho container, và giới hạn `max_connect_errors` để tránh khoá oan khi mạng chập chờn.

2.3.2. Nguyên tắc phân quyền tối thiểu

Script `mysql/init/01-create-users.sh` chạy tự động trong lần khởi tạo cơ sở dữ liệu đầu tiên, tạo thêm hai tài khoản ngoài tài khoản ứng dụng:

- â€¢ `report_ro`: chỉ có quyền `SELECT` và `SHOW VIEW` trên cơ sở dữ liệu `shop_db`. Dùng cho mục đích báo cáo, xuất dữ liệu hoặc phân tích; không thể sửa hay xoá dữ liệu.
- â€¢ `exporter`: có `PROCESS`, `REPLICATION CLIENT` và `SELECT` — đúng ba quyền mysqld-exporter cần để đọc trạng thái máy chủ, kèm `MAX_USER_CONNECTIONS 3` để exporter không bao giờ chiếm nhiều kết nối.

Ba tài khoản (ứng dụng, chỉ đọc, exporter) đều có mật khẩu riêng sinh ngẫu nhiên. Tài khoản `root` chỉ dùng cho tác vụ quản trị khẩn cấp và sao lưu. Cách phân chia này bảo đảm nếu một thành phần bị lộ thông tin đăng nhập, phạm vi thiệt hại được giới hạn.

2.3.3. phpMyAdmin

phpMyAdmin chạy ở mạng nội bộ, không publish cổng, chỉ truy cập được qua Nginx tại đường dẫn con `/phpmyadmin/`. Cấu hình `PMA_ABSOLUTE_URI` được đặt đúng tiền tố để các tệp tĩnh và cookie phiên hoạt động chính xác khi chạy sau proxy:

```
environment:
  PMA_HOST: mysql
  PMA_PORT: 3306
  PMA_ABSOLUTE_URI: "https://${SERVER_NAME}/phpmyadmin/"
  UPLOAD_LIMIT: 64M
  MAX_EXECUTION_TIME: 300
```

Nginx xử lý đường dẫn con bằng cách cắt tiền tố trước khi chuyển tiếp:

```
location /phpmyadmin/ {
    set $phpmyadmin http://phpmyadmin:80;
    rewrite ^/phpmyadmin/(.*)$ /$1 break;
    proxy_pass $phpmyadmin;
    ...
}
```

2.3.4. Cài đặt WordPress không cần trình duyệt

WordPress chưa có cơ sở dữ liệu cấu hình sau khi container khởi động lần đầu; truy cập trang chủ sẽ bị chuyển hướng tới trình cài đặt. Trên máy chủ không có giao diện đồ hoạ, script `scripts/install-wordpress.sh` thực hiện cài đặt qua HTTP: gửi biểu

mẫu tới `/wp-admin/install.php?step=2` với tên website, tài khoản quản trị và mật khẩu lấy từ `.env`, sau đó kiểm tra mã trả về và sự tồn tại của trang đăng nhập.

Tệp cấu hình WordPress được bổ sung qua biến `WORDPRESS_CONFIG_EXTRA` ngay từ lần tạo container, nhờ đó các thiết lập bảo mật có hiệu lực trước cả khi cài đặt:

```
define('WP_HOME', 'https://shop.local');
define('WP_SITEURL', 'https://shop.local');
define('FORCE_SSL_ADMIN', true);      // Mọi thao tác quản trị bắt buộc HTTPS
define('DISALLOW_FILE_EDIT', true);   // Cấm sửa mã nguồn trong dashboard
define('WP_AUTO_UPDATE_CORE', false); // Cập nhật có kiểm soát, không tự động
```

Hai hằng số `WP_HOME` và `WP_SITEURL` tránh lỗi kinh điển khi WordPress nằm sau reverse proxy: nếu không khai báo, WordPress sinh URL theo tiêu đề `Host` và giao thức nhận được, dẫn tới vòng lặp chuyển hướng hoặc tài nguyên tải qua HTTP trên trang HTTPS. `DISALLOW_FILE_EDIT` loại bỏ trình soạn mã trong dashboard — một đường tấn công phổ biến khi tài khoản quản trị bị chiếm.

2.4. Kiểm thử tự động và pipeline CI/CD

Kho mã nguồn có pipeline GitHub Actions tại `.github/workflows/ci.yml`, chạy khi push lên nhánh chính và khi tạo pull request. Pipeline gồm sáu nhóm kiểm tra độc lập:

Bảng 2.2. Các bước kiểm tra trong pipeline

#	Bước	Công cụ và mục đích
1	Kiểm tra cú pháp Compose	`docker compose config -q` — phát hiện sai YAML, biến môi trường thiếu, anchor sai.
2	Build image Nginx	`docker build` — bảo đảm Dockerfile và tệp cấu hình hợp lệ.
3	Kiểm tra cấu hình Nginx	`nginx -t` với chứng chỉ giả — bắt lỗi cú pháp trước khi triển khai.
4	Kiểm tra Prometheus	`promtool check config` và `promtool check rules` — xác thực tệp scrape và quy tắc cảnh báo.
5	Phân tích shell script	`shellcheck` — phát hiện lỗi tiềm ẩn trong tám script vận hành.
6	Quét bí mật cấu hình	`grep` tìm mẫu mật khẩu/khoá bị commit nhầm vào kho.

Bước thứ ba là bài học từ thực tế: một lỗi thiếu dấu chấm phẩy trong `default.conf.template` chỉ làm Nginx từ chối khởi động trên máy chủ, trong khi `nginx -t` phát hiện ngay trong vài giây ở môi trường CI. Bước thứ sáu bảo vệ nguyên tắc quan trọng nhất của dự án: **không có giá trị bí mật nào nằm trong Git**. Tệp

`.env.example` chỉ liệt kê tên biến, còn `.env` thật do `scripts/init-env.sh` sinh trên máy chủ với mật khẩu ngẫu nhiên 32 ký tự hex và quyền `600`.

2.5. Triển khai, kiểm thử và xử lý sự cố thực tế

Quy trình triển khai từ máy mới gồm bốn bước, được gói trong `scripts/deploy.sh`:

1. `./scripts/init-env.sh` — sinh `.env` và `mysqld-exporter/.my.cnf` với mật khẩu ngẫu nhiên, đặt quyền 600.
2. `./scripts/generate-certs.sh` — tạo chứng chỉ tự ký có SAN cho `shop.local`, `localhost` và `127.0.0.1`, hiệu lực 825 ngày.
3. `docker compose up -d --build` — tạo mạng, volume, build Nginx và khởi động mười container theo đúng thứ tự phụ thuộc.
4. `./scripts/install-wordpress.sh` — hoàn tất cài đặt WordPress qua HTTP.

Sau khi triển khai, `scripts/healthcheck.sh` kiểm tra toàn bộ hệ thống và trả về mã lỗi khác 0 nếu bất kỳ mục nào thất bại, nên có thể dùng trong cron hoặc pipeline. Kết quả chạy tại thời điểm nghiệm thu:

```
== Endpoint qua Nginx (Host: shop.local) ==
  [OK]   HTTP -> HTTPS          /                    -> 301
  [OK]   Website                /                    -> 200
  [OK]   wp-admin               /wp-admin/           -> 302
  [OK]   Health endpoint        /healthz             -> 200
  [OK]   phpMyAdmin             /phpmyadmin/         -> 200
  [OK]   Grafana                /grafana/login       -> 200
== Prometheus (localhost) ==
  [OK]   Prometheus healthy -> 200
== Prometheus targets ==
        4 "health":"up"
KET QUA: TAT CA DAT
```

Quá trình đưa hệ thống từ mã nguồn tới trạng thái trên gặp năm sự cố thực tế. Bảng 2.3 ghi lại nguyên nhân và cách khắc phục, vì đây là phần kiến thức chỉ có được khi triển khai thật.

Bảng 2.3. Sự cố gặp phải và cách xử lý

#	Hiện tượng	Nguyên nhân	Cách khắc phục
1	mysqld-exporter khởi động rồi thoát liên tục, log báo "no user specified in section or parent"	Từ phiên bản 0.16, exporter bỏ hỗ trợ biến `DATA_SOURCE_NAME`; bản 0.20 yêu cầu tệp cấu hình kiểu client MySQL.	Chuyển sang `--config.my-cnf=/etc/mysqld-exporter/.my.cnf`; script `init-env.sh` sinh tệp này với quyền 600.

2	Exporter vẫn lỗi sau khi có tệp cấu hình: "permission denied"	Tệp thuộc uid 1000 với quyền 600, còn container chạy bằng user `nobody` nên không đọc được.	Thêm `user: "1000:1000"` cho dịch vụ để tiến trình chạy đúng chủ sở hữu tệp.
3	Sau khi tạo lại container Grafana, Nginx trả về 502	Nginx phân giải tên `grafana` một lần khi nạp cấu hình và giữ địa chỉ IP cũ; container mới có IP khác.	Thêm `resolver 127.0.0.11 valid=10s ipv6=off` và dùng biến cho `proxy_pass`, buộc phân giải lại theo chu kỳ 10 giây.
4	Grafana không khởi động: "Datasource provisioning error: data source not found"	Dashboard tham chiếu datasource theo tên, trong khi provisioning tạo UID khác; dữ liệu cũ trong volume xung đột với cấu hình mới.	Gán UID cố định `prometheus-main`, `loki-main` trong tệp provisioning và trong JSON dashboard; xoá volume Grafana để nạp lại sạch.
5	Tệp sao lưu `wpdata` thuộc sở hữu root, không đổi được quyền	Lệnh `docker run` mặc định chạy bằng root nên tệp tạo ra trong thư mục gắn ngoài thuộc root.	Thêm `--user "$(id -u):$(id -g)"` khi chạy container phụ trợ để tệp thuộc đúng người triển khai.

Về cơ chế hoàn tác, do toàn bộ trạng thái nằm trong Git và volume, quy trình rollback gồm: dừng stack bằng `docker compose down`, chuyển mã nguồn về commit ổn định trước đó bằng `git checkout <commit>`, rồi `docker compose up -d`. Dữ liệu trong volume được giữ nguyên nên chỉ cần phục hồi cơ sở dữ liệu từ bản sao lưu gần nhất khi thay đổi có liên quan tới lược đồ. Trước mỗi lần nâng cấp, `backup.sh` được chạy để bảo đảm luôn có điểm quay lại.

PHẦN 3: TRIỂN KHAI NGINX, HTTPS, BẢO MẬT VÀ SAO LƯU

3.1. Nginx reverse proxy và HTTPS

Nginx là điểm vào duy nhất của hệ thống, đảm nhiệm ba vai trò: chấm dứt TLS, định tuyến yêu cầu tới đúng dịch vụ nội bộ, và ghi log truy cập phục vụ phân tích. Image được build từ `nginx:alpine` để giữ kích thước nhỏ (62,4 MB) và loại bỏ cấu hình mặc định của image gốc.

Cấu hình được sinh từ mẫu bằng `envsubst` ngay khi container khởi động, nhờ đó cùng một tệp mẫu dùng được cho nhiều môi trường chỉ bằng cách đổi biến `SERVER_NAME`:

```
server {
    listen 80;
    server_name ${SERVER_NAME};
    location = /healthz { access_log off; return 200 "ok\n"; }
    location / { return 301 https://$host$request_uri; }
}

server {
    listen 443 ssl;
    http2 on;
    ssl_certificate     /etc/nginx/certs/server.crt;
    ssl_certificate_key /etc/nginx/certs/server.key;
    ssl_protocols       TLSv1.2 TLSv1.3;
    ...
}
```

Chứng chỉ tự ký được sinh bằng `openssl` với phần mở rộng `subjectAltName` gồm `DNS:shop.local`, `DNS:localhost` và `IP:127.0.0.1`. Trên Ubuntu 18.04, OpenSSL 1.1.0 chưa hỗ trợ tham số `-addext`, nên script tạo một tệp cấu hình tạm và truyền qua `-extensions v3`. Thiếu SAN là nguyên nhân phổ biến khiến trình duyệt hiện đại vẫn cảnh báo dù chứng chỉ hợp lệ, vì chúng không còn đọc trường `Common Name`.

Ba đường dẫn được định tuyến tới ba upstream khác nhau, mỗi upstream dùng biến để buộc Nginx phân giải lại DNS nội bộ của Docker:

Bảng 3.1. Bảng định tuyến của Nginx

Đường dẫn	Upstream	Xử lý
/	wordpress:80	Toàn bộ website công khai và trang quản trị. Thời gian chờ 120 giây cho thao tác tải ảnh dung lượng lớn.

/phpmyadmin/	phpmyadmin:80	Cắt tiền tố bằng `rewrite` rồi chuyển tiếp; thời gian chờ 300 giây cho truy vấn nhập/xuất dữ liệu.
/grafana/	grafana:3000	Chuyển tiếp nguyên đường dẫn (Grafana cấu hình `serve_from_sub_path`); hỗ trợ WebSocket cho tính năng Live.

```
resolver 127.0.0.11 valid=10s ipv6=off;

location / {
    set $wordpress http://wordpress:80;
    proxy_pass $wordpress;
    proxy_http_version 1.1;
    proxy_set_header Host              $host;
    proxy_set_header X-Real-IP         $remote_addr;
    proxy_set_header X-Forwarded-For   $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}
```

Việc dùng biến trong `proxy_pass` là điểm kỹ thuật quan trọng. Khi `proxy_pass` nhận tên miền cố định, Nginx phân giải DNS đúng một lần lúc nạp cấu hình và giữ nguyên địa chỉ IP đó. Trong môi trường Docker, mỗi lần tạo lại container upstream, địa chỉ IP thay đổi, khiến Nginx chuyển tiếp tới địa chỉ không còn tồn tại và trả về 502. Kết hợp `resolver 127.0.0.11` (DNS nội bộ của Docker) với biến và `valid=10s`, Nginx tự phân giải lại sau mỗi 10 giây, loại bỏ hoàn toàn lớp sự cố này. Đây chính là sự cố số 3 trong Bảng 2.3.

Ngoài định tuyến, Nginx còn thiết lập các tiêu đề bảo mật và tối ưu:

```
add_header Strict-Transport-Security "max-age=86400" always;
add_header X-Frame-Options           "SAMEORIGIN" always;
add_header X-Content-Type-Options    "nosniff" always;
add_header Referrer-Policy           "strict-origin-when-cross-origin" always;
add_header Permissions-Policy        "geolocation=(), microphone=(), camera=()"
always;
client_max_body_size 64m;
```

`Strict-Transport-Security` buộc trình duyệt dùng HTTPS trong các lần truy cập sau, chống hạ cấp giao thức. `X-Frame-Options: SAMEORIGIN` ngăn nhúng website vào khung của miền khác, hạn chế tấn công clickjacking. `nosniff` chặn trình duyệt tự đoán kiểu nội dung — một đường để biến tệp tải lên thành mã script. `client_max_body_size 64m` cho phép tải ảnh sản phẩm chất lượng cao nhưng vẫn chặn tệp quá lớn.

Cấu hình toàn cục `nginx.conf` tắt `server_tokens` để không lộ phiên bản Nginx trong phản hồi và tiêu đề lỗi, đồng thời định nghĩa định dạng log có thời gian xử lý để phục vụ phân tích hiệu năng qua Loki:

```
log_format main '$remote_addr - $remote_user [$time_local] "$request" '
                '$status $body_bytes_sent "$http_referer" '
                '"$http_user_agent" rt=$request_time
urt=$upstream_response_time';
```

3.2. Tăng cường bảo mật máy chủ và ứng dụng

Hardening được thực hiện theo nguyên tắc phòng thủ nhiều lớp: mỗi lớp giả định lớp trước có thể bị vượt qua. Bảng 3.2 tổng hợp các biện pháp đã áp dụng và lý do.

Bảng 3.2. Các biện pháp tăng cường bảo mật đã triển khai

#	Biện pháp	Phạm vi	Mục đích
1	`no-new-privileges:true`	Toàn bộ 10 container	Chặn leo thang đặc quyền qua tệp setuid bên trong container.
2	`internal: true` cho mạng backend	MySQL và tầng dữ liệu	Không có route ra Internet; dữ liệu không thể bị đưa ra ngoài kể cả khi ứng dụng bị chiếm.
3	Không publish cổng MySQL, phpMyAdmin, Loki	Port mapping	Chỉ truy cập được từ trong mạng Docker hoặc qua Nginx đã xác thực TLS.
4	Prometheus bind `127.0.0.1:9090`	Prometheus	Chỉ truy vấn được từ chính máy chủ hoặc qua SSH tunnel, không lộ ra mạng LAN.
5	Ba tài khoản CSDL riêng, quyền tối thiểu	MySQL	Giới hạn thiệt hại khi lộ một bộ thông tin đăng nhập; exporter bị giới hạn 3 kết nối.
6	`DISALLOW_FILE_EDIT`	WordPress	Loại bỏ trình soạn mã trong dashboard — đường duy trì truy cập của kẻ tấn công.
7	`FORCE_SSL_ADMIN`, HSTS	WordPress, Nginx	Không cho phép đăng nhập/cookie quản trị đi qua kênh không mã hoá.
8	`server_tokens off`	Nginx	Không tiết lộ phiên bản phần mềm để đối chiếu lỗ hổng đã biết.
9	TLS 1.2/1.3, tắt session ticket	Nginx	Loại bỏ giao thức lỗi thời; session ticket tắt để giảm nguy cơ lộ khoá phiên.
10	Bí mật tách khỏi mã nguồn	Toàn dự án	`.env` quyền 600, `.gitignore` chặn commit; CI quét mẫu bí mật.
11	Giới hạn log 10 MB × 3 tệp	Toàn bộ container	Chống tấn công làm đầy ổ đĩa qua ghi log (log flooding).
12	`restart: unless-stopped` + healthcheck	Toàn bộ dịch vụ	Tự phục hồi sau sự cố; giảm thời gian gián đoạn.

Với lớp ứng dụng, WordPress được cấu hình để người dùng quản trị đặt mật khẩu mạnh và không dùng tên đăng nhập mặc định `admin`. Tài khoản quản trị được sinh ngẫu nhiên trong `.env` và giá trị không xuất hiện trong báo cáo hay log. Với lớp mạng,

chỉ hai cổng 80 và 443 được mở ra ngoài; mọi dịch vụ khác chỉ liên lạc trong mạng nội bộ Docker.

Một điểm cần nêu thẳng thắn: hệ thống dùng chứng chỉ tự ký nên trình duyệt sẽ cảnh báo khi truy cập. Trong môi trường học tập điều này chấp nhận được, nhưng khi triển khai thật cần thay bằng chứng chỉ từ tổ chức cấp phát (Let's Encrypt hoặc chứng chỉ thương mại). Script `generate-certs.sh` được viết tách riêng để việc thay thế chỉ là thay hai tệp trong `nginx/certs/` và khởi động lại Nginx.

3.3. Sao lưu và phục hồi

Dữ liệu cần bảo vệ gồm hai phần: cơ sở dữ liệu (nội dung sản phẩm, tài khoản, cấu hình) và thư mục `wp-content` (ảnh sản phẩm, giao diện, plugin). Script `scripts/backup.sh` xử lý cả hai trong một lần chạy.

Với cơ sở dữ liệu, `mysqldump` được gọi với `--single-transaction` để tạo bản sao nhất quán mà không khoá bảng — website vẫn phục vụ khách trong lúc sao lưu. Các tham số `--routines --triggers --events` bảo đảm không bỏ sót thủ tục, trigger và sự kiện định kỳ:

```
docker exec de3-mysql sh -c \
  'exec mysqldump -uroot -p"$MYSQL_ROOT_PASSWORD" \
     --single-transaction --quick --routines --triggers --events \
     "$MYSQL_DATABASE"' | gzip -9 > "$DB_FILE"
chmod 600 "$DB_FILE"
```

Với thư mục upload, script dùng một container phụ trợ gắn volume ở chế độ chỉ đọc và chạy bằng chính uid của người triển khai, nhờ đó tệp nén thuộc đúng chủ sở hữu thay vì root (sự cố số 5 trong Bảng 2.3). Chính sách lưu giữ là bảy bản gần nhất cho mỗi loại, bản cũ hơn bị xoá tự động để không làm đầy ổ đĩa.

Kết quả chạy thực tế tại thời điểm nghiệm thu:

```
-rw------- 1 thinh thinh  20K Oct 25 10:40 db_shop_db_2026-10-25_104056.sql.gz
-rw------- 1 thinh thinh  13M Oct 25 10:40 wpdata_2026-10-25_104056.tar.gz
```

Tệp cơ sở dữ liệu nhỏ vì website mới có nội dung mẫu; tệp `wpdata` 13 MB chủ yếu là mã nguồn WordPress, giao diện và thư viện, phần nội dung thật sẽ tăng dần theo ảnh sản phẩm được tải lên. Cả hai tệp có quyền 600, chỉ người triển khai đọc được.

Script `scripts/restore.sh` thực hiện chiều ngược lại: giải nén thư mục upload vào volume, sau đó nạp bản dump vào MySQL. Quy trình phục hồi được khuyến nghị gồm bốn bước: dừng WordPress để không có ghi mới, phục hồi dữ liệu, khởi động lại và chạy `healthcheck.sh` để xác nhận. Việc lên lịch được thực hiện bằng cron trên máy chủ:

```
# Sao lưu hằng ngày lúc 02:00
0 2 * * * cd /home/thinh/de3-wordpress-quangba && ./scripts/backup.sh >>
backups/backup.log 2>&1
```

Để bảo vệ trước sự cố mất toàn bộ máy ảo, bản sao lưu cần được đồng bộ ra ngoài. Trong phạm vi học phần, điều này được nêu như khuyến nghị vận hành (đồng bộ lên máy chủ lưu trữ khác hoặc dịch vụ đối tượng) chứ chưa triển khai, vì máy ảo chỉ có một ổ đĩa 20 GB.

PHẦN 4: GIÁM SÁT VÀ THU THẬP LOG TẬP TRUNG

4.1. Prometheus và các chỉ số thu thập

Prometheus hoạt động theo mô hình kéo (pull): định kỳ gọi endpoint `/metrics` của từng mục tiêu, lưu kết quả dưới dạng chuỗi thời gian và cho phép truy vấn bằng PromQL. Hệ thống cấu hình bốn job thu thập, mỗi job đại diện một lớp thông tin khác nhau:

Bảng 4.1. Các mục tiêu Prometheus thu thập

Job	Endpoint	Nhóm chỉ số
prometheus	localhost:9090	Sức khoẻ bản thân Prometheus, số mẫu đã thu, thời gian scrape.
node-exporter	node-exporter:9100	CPU theo từng chế độ, bộ nhớ, dung lượng và inode ổ đĩa, lưu lượng mạng, tải hệ thống.
mysqld-exporter	mysqld-exporter:9104	Số kết nối, số truy vấn, bộ đệm, khoá, tình trạng replication.
loki	loki:3100	Sức khoẻ Loki, số luồng log đang quản lý, tốc độ nhận log.

Thời gian lưu trữ được đặt 15 ngày (`--storage.tsdb.retention.time=15d`), đủ để so sánh xu hướng theo tuần mà vẫn phù hợp ổ đĩa 20 GB. Dữ liệu nằm trong volume `prometheus_data` nên không mất khi tạo lại container.

Ngoài việc thu thập, Prometheus đánh giá tám quy tắc cảnh báo chia ba nhóm, giúp phát hiện sự cố trước khi người dùng phản ánh:

Bảng 4.2. Các quy tắc cảnh báo

Nhóm	Cảnh báo	Điều kiện kích hoạt
availability	TargetDown	Một mục tiêu không scrape được quá 1 phút (`up == 0`).
PrometheusConfigReloadFailed	Nạp lại cấu hình thất bại — cấu hình mới không có hiệu lực.	
host-resources	HighCpuUsage	CPU trung bình 5 phút vượt 85%.
HighMemoryUsage	RAM khả dụng dưới 10% trong 10 phút.	
LowDiskSpace	Dung lượng trống dưới 15%.	
application	MySQLDown	Không thu được chỉ số MySQL trong 1 phút.

MySQLTooManyConnections	Số kết nối vượt 80% giới hạn `max_connections`.
MySQLSlowQueriesIncreasing	Tốc độ truy vấn chậm tăng liên tục — dấu hiệu thiếu chỉ mục.

Các quy tắc được kiểm tra cú pháp bằng `promtool check rules` trong pipeline CI, và ở thời điểm nghiệm thu cả tám cảnh báo đều ở trạng thái `inactive` — nghĩa là hệ thống đang khoẻ mạnh và bộ quy tắc đã nạp thành công. Ngưỡng cảnh báo được chọn theo kinh nghiệm vận hành: cảnh báo quá nhạy sẽ gây mỏi mệt và bị bỏ qua, quá trễ thì mất tác dụng.

4.2. Grafana và dashboard

Grafana là lớp trực quan hoá, kết nối tới Prometheus cho số liệu và Loki cho log. Điểm quan trọng về mặt quản trị là toàn bộ cấu hình được **provisioning** — nghĩa là datasource và dashboard được khai báo bằng tệp YAML/JSON và nạp tự động khi khởi động, không phải tạo tay qua giao diện. Nhờ đó, dựng lại hệ thống trên máy khác chỉ cần chạy lại stack là có đủ dashboard, không phụ thuộc vào thao tác thủ công của một cá nhân.

```
datasources:
  - name: Prometheus
    uid: prometheus-main
    type: prometheus
    url: http://prometheus:9090
    isDefault: true
  - name: Loki
    uid: loki-main
    type: loki
    url: http://loki:3100
```

UID được cố định (`prometheus-main`, `loki-main`) thay vì để Grafana sinh ngẫu nhiên. Lý do xuất phát từ sự cố số 4: tệp JSON của dashboard tham chiếu datasource theo UID, nếu UID thay đổi giữa các lần dựng lại thì mọi panel sẽ báo lỗi "data source not found". Đặt UID cố định biến dashboard thành tệp cấu hình thực sự có thể tái sử dụng.

Dashboard **"De 3 - Tong quan he thong"** gồm bảy panel, tự làm mới mỗi 30 giây:

Bảng 4.3. Các panel của dashboard

#	Panel	Chỉ số và ngưỡng màu

1	CPU sử dụng	`100 - avg(rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100`; xanh < 70%, cam 70–85%, đỏ > 85%.
2	RAM sử dụng	`(1 - MemAvailable/MemTotal) * 100`; cam từ 80%, đỏ từ 90%.
3	Ổ đĩa còn trống	Tỉ lệ `filesystem_avail / filesystem_size` của `/`; đỏ dưới 15%.
4	CPU & RAM theo thời gian	Hai chuỗi thời gian cùng trục phần trăm để thấy tương quan khi tải tăng.
5	MySQL — kết nối & truy vấn/giây	`mysql_global_status_threads_connected` và `rate(mysql_global_status_queries[5m])`.
6	Lưu lượng mạng	Tốc độ nhận byte theo giao diện mạng, loại trừ `lo` và các giao diện ảo của Docker.
7	Log Nginx (Loki)	Truy vấn LogQL `{container="de3-nginx"}`, hiển thị log mới nhất ở trên.

Panel thứ bảy là điểm nối giữa hai hệ thống giám sát: cùng một màn hình vừa có số liệu định lượng (CPU, RAM) vừa có log định tính. Khi CPU tăng đột biến, người quản trị đọc ngay được dòng log tương ứng để biết đó là do lượng truy cập thật hay do một yêu cầu bất thường.

Grafana được cấu hình chạy sau đường dẫn con `/grafana/` với `GF_SERVER_ROOT_URL` và `GF_SERVER_SERVE_FROM_SUB_PATH=true`. Ba thiết lập bảo mật được bật: tắt đăng ký tự do (`GF_USERS_ALLOW_SIGN_UP=false`), tắt truy cập ẩn danh (`GF_AUTH_ANONYMOUS_ENABLED=false`) và tắt gửi số liệu phân tích (`GF_ANALYTICS_REPORTING_ENABLED=false`). Như vậy chỉ tài khoản quản trị trong `.env` mới xem được dashboard.

4.3. Thu thập log tập trung với Loki và Promtail

Loki đóng vai trò tương tự Prometheus nhưng cho log: thay vì đánh chỉ mục toàn bộ nội dung log (tốn dung lượng và bộ nhớ như Elasticsearch), Loki chỉ đánh chỉ mục theo nhãn và lưu phần nội dung dưới dạng khối nén. Cách tiếp cận này phù hợp với máy ảo 4 GB RAM, đổi lại truy vấn nội dung chậm hơn một chút so với tìm kiếm toàn văn.

Cấu hình Loki dùng schema phiên bản 13 với chỉ mục `tsdb`, lưu trữ trên hệ thống tệp trong volume `loki_data`, thời gian giữ 168 giờ (bảy ngày) và bật compactor để dọn

dữ liệu cũ đúng hạn. Mức bảy ngày được chọn để đủ điều tra sự cố trong một chu kỳ vận hành mà không chiếm quá nhiều đĩa.

Promtail là agent chạy trên máy chủ, đọc log của container qua Docker API. Thay vì liệt kê cứng từng container, cấu hình dùng cơ chế khám phá động `docker_sd_configs` và lọc theo nhãn dự án:

```
scrape_configs:
  - job_name: docker
    docker_sd_configs:
      - host: unix:///var/run/docker.sock
    relabel_configs:
      - source_labels:
['__meta_docker_container_label_com_docker_compose_project']
        regex: 'de3-wordpress'
        action: keep
      - source_labels: ['__meta_docker_container_name']
        regex: '/(.*)'
        target_label: 'container'
      - source_labels:
['__meta_docker_container_label_com_docker_compose_service']
        target_label: 'service'
      - source_labels:
['__meta_docker_container_label_com_docker_compose_project']
        target_label: 'project'
```

Nhãn `keep` đảm bảo chỉ log của dự án `de3-wordpress` được thu, không lẫn log hệ thống của các container khác trên máy. Ba nhãn `container`, `service`, `project` tạo thành hệ toạ độ để truy vấn: theo tên container cụ thể, theo loại dịch vụ, hoặc toàn bộ dự án.

LogQL là ngôn ngữ truy vấn của Loki, kết hợp chọn luồng (stream selector) trong ngoặc nhọn với bộ lọc chuỗi và hàm tổng hợp. Một số truy vấn đã dùng trong quá trình kiểm chứng:

Bảng 4.4. Các truy vấn LogQL tiêu biểu

Truy vấn	Mục đích
`{container="de3-nginx"}`	Xem toàn bộ log truy cập Nginx.
`{container="de3-nginx"} |= " 500 "`	Lọc các phản hồi lỗi máy chủ để điều tra.
`{container="de3-nginx"} |~ " 4[0-9][0-9] "`	Tìm lỗi phía người dùng (404, 403…) bằng biểu thức chính quy.
`sum by (container) (rate({project="de3-wordpress"}[5m]))`	Đếm số dòng log mỗi giây theo container — phát hiện container ghi log bất thường.
`{container="de3-nginx"} |= "rt=" | pattern "<_> rt=<rt>"`	Trích thời gian xử lý từ định dạng log tuỳ biến để phân tích độ trễ.

Vì image Promtail không có sẵn `wget` hay `curl`, việc kiểm chứng Loki được thực hiện qua API proxy của Grafana (`/api/datasources/proxy/uid/loki-main/loki/api/v1/...`). Cách này còn có ưu điểm dùng đúng đường đi mà người vận hành thật sẽ dùng. Kết quả kiểm chứng cho thấy Loki trả về danh sách nhãn đầy đủ và truy vấn `{container="de3-nginx"}` trả về các dòng log thật của Nginx, xác nhận toàn bộ chuỗi Promtail → Loki → Grafana hoạt động thông suốt.

4.4. Đánh giá hệ thống giám sát và kết quả nghiệm thu

Bảng 4.5 tổng hợp kết quả đo trực tiếp trên máy ảo tại thời điểm nghiệm thu ngày 25/10/2026, sau khi hệ thống chạy ổn định.

Bảng 4.5. Kết quả nghiệm thu

#	Hạng mục	Kết quả
1	Số container đang chạy	10/10 `Up`; MySQL, WordPress và Nginx ở trạng thái `healthy`
2	Cổng công bố	80, 443 (Nginx); 127.0.0.1:9090 (Prometheus) — các dịch vụ khác không publish cổng
3	Website	`https://shop.local/` → 200; `/wp-admin/` → 302 (chuyển tới đăng nhập)
4	Chuyển hướng HTTP → HTTPS	301, đúng như thiết kế
5	phpMyAdmin	`/phpmyadmin/` → 200
6	Grafana	`/grafana/login` → 200, phiên bản 13.2.2
7	Prometheus targets	4/4 `up` (prometheus, node-exporter, mysqld-exporter, loki)
8	Quy tắc cảnh báo	8 quy tắc đã nạp, tất cả `inactive`
9	PromQL — CPU	≈ 39–43% (2 vCPU)
10	PromQL — RAM	≈ 58,7% trên 4,1 GB
11	PromQL — kết nối MySQL	2 kết nối đang mở
12	LogQL	`{container="de3-nginx"}` trả về log thật, nhãn `container/service/project` đầy đủ
13	Sao lưu	`db_shop_db_*.sql.gz` (20 KB) và `wpdata_*.tar.gz` (13 MB), quyền 600
14	Kết quả script kiểm tra	`KET QUA: TAT CA DAT`

Về hiệu năng, mức dùng CPU 39–43% và RAM 58,7% cho thấy hệ thống còn dư địa nhưng không thoải mái: mười container trên 4,1 GB RAM là cấu hình sát ngưỡng hợp lý. Nếu tăng thêm thành phần (ví dụ WooCommerce cùng các bảng dữ liệu lớn),

cần nâng RAM lên 8 GB hoặc giảm thời gian lưu chỉ số. Đây là đánh giá định lượng có được nhờ dữ liệu Prometheus, minh hoạ đúng giá trị của giám sát: quyết định nâng cấp dựa trên số đo chứ không dựa trên cảm nhận.

Hệ thống giám sát hiện tại có ba ưu điểm: (i) phủ đủ ba lớp hạ tầng – hệ điều hành – ứng dụng; (ii) cấu hình hoàn toàn bằng mã nên tái lập được; (iii) log và metric cùng hiển thị trên một dashboard. Hạn chế cần nêu: chưa có kênh gửi cảnh báo tự động (email, Telegram) nên quy tắc cảnh báo hiện chỉ hiển thị trong Prometheus/Grafana mà chưa đẩy thông báo tới người vận hành; chưa có dashboard riêng cho Loki để phân tích xu hướng log dài hạn; và các chỉ số về trải nghiệm người dùng (thời gian tải trang phía client) chưa được thu thập. Ba hạn chế này được đưa vào hướng phát triển.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Báo cáo đã trình bày quá trình xây dựng và triển khai hoàn chỉnh hệ thống Website Quảng bá Sản phẩm bằng WordPress trên máy ảo Ubuntu 18.04.6 LTS, đáp ứng đầy đủ các yêu cầu của Đề 3. Hệ thống gồm mười container chạy trên ba vùng mạng được cách ly, sáu volume lưu trữ bền vững, một reverse proxy Nginx chấm dứt TLS cho ba dịch vụ web, cùng bộ công cụ giám sát Prometheus – Grafana – Loki – Promtail. Toàn bộ cấu hình nằm trong kho Git với pipeline CI kiểm tra sáu bước.

Kết quả chính đạt được:

- â€¢ **Website hoạt động đầy đủ**: trang công khai trả về 200 qua HTTPS, trang quản trị yêu cầu đăng nhập đúng thiết kế, HTTP tự chuyển hướng sang HTTPS.
- â€¢ **Kiến trúc tách biệt rõ ràng**: tầng dữ liệu nằm trong mạng `internal` không có route ra Internet; chỉ Nginx mở cổng 80/443; Prometheus chỉ bind trên localhost.
- â€¢ **Quản trị cơ sở dữ liệu an toàn**: MySQL 8.0 với utf8mb4, ba tài khoản phân quyền tối thiểu, phpMyAdmin truy cập qua Nginx.
- â€¢ **Giám sát đầy đủ hai trụ cột**: 4/4 mục tiêu Prometheus hoạt động, 8 quy tắc cảnh báo đã nạp, dashboard 7 panel tự làm mới 30 giây, log Nginx truy vấn được bằng LogQL.
- â€¢ **Vận hành có kỷ luật**: script kiểm tra sức khoẻ tự động, sao lưu hằng ngày giữ 7 bản, quy trình rollback dựa trên Git và volume.

Giá trị lớn nhất của quá trình thực hiện không nằm ở việc hệ thống chạy được, mà ở năm sự cố thực tế đã gặp và khắc phục (Bảng 2.3). Mỗi sự cố để lại một bài học cụ thể: `mysql -t` kiểm tra được cú pháp trước khi triển khai; Nginx giữ DNS cũ khi container được tạo lại; image mới có thể bỏ tham số cũ mà không báo trước; quyền truy cập tệp phải khớp với user chạy trong container. Đây là những kinh nghiệm không có trong tài liệu hướng dẫn và chỉ xuất hiện khi triển khai trên môi trường thật.

Hướng phát triển được xác định từ các hạn chế đã nêu:

1. **Bổ sung kênh cảnh báo tự động**: cấu hình Alertmanager gửi thông báo qua email hoặc Telegram khi có cảnh báo nghiêm trọng, kèm cơ chế im lặng (silence) khi bảo trì.
2. **Mở rộng nội dung sản phẩm**: cài WooCommerce để có danh mục, giỏ hàng và thanh toán; hoặc xây dựng kiểu nội dung tuỳ biến (custom post type) cho sản phẩm với các trường giá, bảo hành, thông số.
3. **Tự động hoá triển khai**: bổ sung job CD trong GitHub Actions kết nối SSH tới máy ảo để pull code và chạy `deploy.sh` khi nhánh chính thay đổi.
4. **Tăng cường bảo mật nâng cao**: thay chứng chỉ tự ký bằng Let's Encrypt với tự động gia hạn, bật WAF (ModSecurity hoặc fail2ban) chặn quét cổng và dò mật khẩu, bổ sung sao lưu ra ngoài máy ảo.
5. **Trực quan hoá log nâng cao**: tạo dashboard Loki riêng hiển thị tỉ lệ mã phản hồi HTTP theo thời gian, các endpoint chậm và lưu lượng theo giờ, phục vụ phân tích hành vi người dùng.
6. **Mở rộng quy mô**: khi lưu lượng tăng, có thể nâng lên 8 GB RAM và tách máy chủ giám sát khỏi máy chủ ứng dụng, đúng mô hình mà ba vùng mạng đã chuẩn bị sẵn về mặt logic.

Nhìn tổng thể, đề tài đã hoàn thành mục tiêu trở thành một hệ thống hoàn chỉnh ở mức có thể vận hành thật: tái lập được bằng một lệnh, quan sát được bằng dashboard, bảo vệ được bằng nhiều lớp cấu hình và có đường lùi khi xảy ra sự cố. Đó cũng là nội dung cốt lõi của học phần Triển khai và Quản trị Hệ thống Phần mềm.

TÀI LIỆU THAM KHẢO

[1] Docker Inc., *Docker Compose file reference*. https://docs.docker.com/compose/compose-file/ (truy cập 25/10/2026).

[2] Docker Inc., *Docker networking — bridge drivers and internal networks*. https://docs.docker.com/network/ (truy cập 25/10/2026).

[3] Nginx Inc., *NGINX documentation: reverse proxy, SSL/TLS, resolver*. https://nginx.org/en/docs/ (truy cập 25/10/2026).

[4] WordPress.org, *WordPress documentation: Installation, wp-config.php, Database description*. https://wordpress.org/documentation/ (truy cập 25/10/2026).

[5] Oracle, *MySQL 8.0 Reference Manual: Security, Authentication plugins, mysqldump*. https://dev.mysql.com/doc/refman/8.0/en/ (truy cập 25/10/2026).

[6] Prometheus Authors, *Prometheus documentation: Configuration, PromQL, Alerting rules*. https://prometheus.io/docs/ (truy cập 25/10/2026).

[7] Grafana Labs, *Grafana documentation: Provisioning, Data sources, Dashboard JSON model*. https://grafana.com/docs/grafana/latest/ (truy cập 25/10/2026).

[8] Grafana Labs, *Loki documentation: Configuration, LogQL, Promtail docker_sd_configs*. https://grafana.com/docs/loki/latest/ (truy cập 25/10/2026).

[9] OWASP Foundation, *OWASP Top 10 — 2021*. https://owasp.org/Top10/ (truy cập 25/10/2026).

[10] OWASP Foundation, *OWASP Docker Security Cheat Sheet*. https://cheatsheetseries.owasp.org/ (truy cập 25/10/2026).

[11] GitHub, *GitHub Actions documentation: Workflow syntax, jobs, steps*. https://docs.github.com/en/actions (truy cập 25/10/2026).

[12] Canonical, *Ubuntu Server Guide — Ubuntu 18.04 LTS*. https://ubuntu.com/server/docs (truy cập 25/10/2026).